

Anil Seth



Static to Drupal 7

Save time, impress colleagues, switch to a CMS in days!

At some point in the life of a static site, you will wish that you had used a CMS. The reasons could be varied:

- You may be suffering from the acute pain of changing the look and feel of the site.
- You may need to make the site look good on smart mobile devices, netbooks, tablets, or even extra-wide desktop screens.
- Your organisation may finally realise that supporting IE 5 and 6 is not enough. You no longer can keep ignoring the growing tribe of Google Chrome and Firefox users.
- You may need to modify the content and keep it current.

It is surprising how often the implications of the last point are underestimated. The department complaining about the content being incorrect, and the department responsible for providing the correct information, are not necessarily the same. It is very hard to delegate the responsibility for modifying static content. You, as the person accountable for maintaining the site, are stuck in the middle, and it can be a very frustrating experience.

Switching to a CMS

As you explore the option of switching to a CMS, you will probably try to estimate the effort of transferring the existing content into the new site. It is not an easy decision. Should you modify the content during the migration? Obviously, the advantages of a CMS will be minimal, unless you reorganise the content based on the organisation's needs and the capabilities of the CMS. This activity takes a long time—most of which is spent in getting everybody to agree on a suitable design. Multiple models may need to be explored. Meanwhile, the backlog on the existing site continues to grow.

It would be great if you could transfer the existing content painlessly into the CMS structure. Then you could replace the pages as and when required, into the ideal or appropriate structure. You could then shorten the period for which you need to maintain both the existing site and the new site being created. The stakeholders would get a better feel of what the site would be like with their own content. The theme and the colour schemes can be decided on while you build the new site. Switching a theme is not hard—if you haven't customised a theme substantially.

Import HTML and Drupal to the rescue

Drupal comes with an Import HTML module, which allows you to import an existing HTML site—see http://drupal.org/project/import_html. It provides a couple of sample templates for you to get started—and the best part is, it is now available for Drupal 7. Try a few pages first. Chances are that your imported content will look like such a mess that you'll think it's hopeless and want to give up.

Don't give up too soon! The templates for transferring the content are in XSLT. It is bad enough to write XML by hand, and very few people I know are comfortable writing XSLT scripts. But persist; it is not as hard as it looks. The samples are sufficient to get started with, and you may need to make only some common changes.

Typically, a page will consist of a header, footer, sidebars and content. You will probably want to manage the first three using Drupal 7. Hence, you can transfer the content from the static pages into content in Drupal 7 pages. *Import_HTML* will create reasonable menu links and preserve the existing page names. It will also import content like PDF files and images into a suitable folder.

Menu-editing features are very nice in Drupal 7, and you can easily reorganise the menus once the content is meaningfully imported.

Unless your site is absolutely chaotic, you will find patterns. At worst, there may be a couple of sub-sites, and you may need to create separate templates for each of them. Start with the *simplehtml2simplehtml.xml*. Here, you will find the template 'get-content'. Just below it, notice the following lines:

```
<xsl:template match="*html:h1">
  <!-- Remove the found H1 from the flow altogether. -->
</xsl:template>
```

This will match the HTML <h1> tag; ignore it in the conversion. You will soon understand why H1 content is excluded.

So, the idea is simple. You need to identify elements to ignore, retaining only the desired content. Here's an example:

(Continued On Page 62....)